Capítulo 01 - Introdução e Visão Geral do Sistema

1.1. Contexto e Motivação

O Projeto Quimera nasceu da intersecção entre três domínios tecnológicos em rápida evolução: **automação web**, **inteligência artificial** e **engenharia de software de sistemas distribuídos**. Em sua essência, o projeto representa uma tentativa de resolver um problema complexo de otimização: como identificar, qualificar e iniciar interações sociais de forma automatizada, personalizada e em larga escala, mantendo um nível de autenticidade que simule interações humanas genuínas.

1.1.1. O Problema da Prospecção Manual

A prospecção manual em plataformas de mídia social é um processo inerentemente ineficiente e que não escala. Considere o seguinte cenário típico:

Um usuário deseja conhecer pessoas com interesses compatíveis em sua região (Varginha, MG). O processo manual envolve:

1. **Identificação de locais relevantes** (hotspots): Bares, academias, eventos culturais, perfis de influenciadores locais
2. **Navegação manual** até cada perfil do Instagram ou Facebook associado a esses locais
3. **Análise visual e textual** de cada perfil: fotos, biografia, postagens recentes
4. **Inferência subjetiva** sobre compatibilidade: interesses, estilo de vida, status de relacionamento
5. **Decisão de contato** baseada em critérios vagos e inconsistentes
6. **Composição de mensagem** personalizada para cada perfil selecionado
7. **Envio manual** da mensagem
8. **Monitoramento** da caixa de entrada para detectar respostas
9. **Continuação** da conversa de forma manual

Este processo, quando executado manualmente, apresenta várias limitações críticas:

- **Baixa throughput:** Um usuário experiente consegue analisar e contatar, no máximo, 10-15 perfis por hora, resultando em ~50-100 perfis por semana.
- **Inconsistência:** Os critérios de seleção variam com o humor, cansaço e vieses cognitivos do momento.

- **Fadiga decisional:** Após analisar dezenas de perfis, a qualidade das decisões degrada significativamente.
- **Custo de oportunidade:** O tempo investido nesta tarefa repetitiva poderia ser alocado em atividades de maior valor.
- **Falta de dados:** Não há registro sistemático das decisões, impossibilitando aprendizado e otimização.

1.1.2. A Solução: Automação Inteligente

O Projeto Quimera propõe uma solução radical: **automatizar completamente** o funil de prospecção, desde a descoberta de perfis até o primeiro contato, utilizando uma arquitetura de **agentes inteligentes** que operam de forma coordenada.

A solução se baseia em três pilares fundamentais:

1. Automação Web Furtiva

Utilizando técnicas avançadas de web scraping e RPA (Robotic Process Automation), o sistema simula o comportamento de um usuário humano navegando pelas plataformas de mídia social. Isso inclui:

- Movimentos de mouse realistas com curvas de Bézier
- Delays variáveis entre ações para simular tempo de leitura e reflexão
- Scrolling gradual de páginas
- Rotação de proxies residenciais para mascarar o endereço IP
- Rotação de User-Agents para simular diferentes dispositivos
- Canvas fingerprint poisoning para evitar identificação única do navegador

2. Inteligência Artificial Multi-Modal

O sistema emprega modelos de IA de última geração para analisar perfis de forma holística:

- **NLP (Natural Language Processing):** Análise de biografias e postagens para extrair interesses, personalidade e contexto social
- **Computer Vision:** Análise de fotos de perfil e postagens para inferir atributos estéticos, estilo de vida e contexto
- **Embeddings Semânticos:** Representação vetorial de perfis para cálculo de similaridade e afinidade
- **LLMs (Large Language Models):** Geração de mensagens personalizadas e autênticas

3. Arquitetura de Agentes Especializados

Em vez de um sistema monolítico, o Projeto Quimera adota uma arquitetura modular baseada em agentes, onde cada agente tem uma responsabilidade clara e bem definida:

- **Agente Batedor:** Prospecção e coleta de URLs de perfis
- **Agente Analista:** Análise profunda e pontuação de perfis
- **Motor de Persuasão:** Desenvolvimento de estratégia de comunicação
- **Agente Escriba:** Geração de mensagens personalizadas
- **Agente de Execução:** Envio automatizado de mensagens
- **Agente de Monitoramento:** Detecção de respostas e acionamento de handoff
- **Agente de Aprendizagem:** Melhoria contínua baseada em feedback

Esta arquitetura permite **escalabilidade horizontal** (adicionar mais instâncias de agentes), **resiliência** (falha de um agente não derruba o sistema) e **manutenibilidade** (cada agente pode ser desenvolvido e testado independentemente).

1.1.3. Resultados Esperados

Quando operando em plena capacidade, o sistema é projetado para:

- **Analisar** 500-1.000 perfis por dia de forma totalmente automatizada
- **Contatar** 50-100 perfis qualificados por dia (após filtragem rigorosa)
- **Manter** uma taxa de resposta de 20-30% (superior à média de 10-15% em abordagens genéricas)
- **Converter** 60-80% das respostas positivas em handoff bem-sucedido (transição para conversa humana)
- **Operar** 24/7 sem intervenção humana, exceto para handoff de conversas promissoras

Isso representa um aumento de **10-20x** na capacidade de prospecção comparado ao processo manual, mantendo ou até superando a qualidade das interações.

1.2. Objetivos do Sistema

1.2.1. Objetivos Funcionais

OF1. Descoberta Automatizada de Perfis

O sistema deve ser capaz de descobrir perfis relevantes de forma autônoma, navegando por hotspots digitais pré-configurados (perfis de estabelecimentos, eventos, influenciadores) e extraindo URLs de perfis que interagiram com esses hotspots.

Critério de Sucesso: Coletar pelo menos 500 URLs de perfis únicos por dia, com taxa de erro < 5%.

OF2. Análise Multi-Dimensional de Perfis

O sistema deve analisar cada perfil coletado em múltiplas dimensões, extraindo informações estruturadas e não-estruturadas para construir um "dossiê digital" completo.

Dimensões de Análise:

- **Textual:** Biografia, postagens, comentários, hashtags
- **Visual:** Fotos de perfil, fotos de postagens, stories
- **Social:** Número de seguidores, seguindo, engajamento, rede de conexões
- **Temporal:** Frequência de postagens, horários de atividade, padrões temporais
- **Geográfica:** Localização declarada, check-ins, geotags em fotos

Critério de Sucesso: Extrair pelo menos 80% das features planejadas para cada perfil, com tempo de processamento < 2 minutos por perfil.

OF3. Pontuação e Ranking Automatizados

O sistema deve atribuir uma pontuação numérica a cada perfil analisado, refletindo o grau de compatibilidade com os critérios definidos pelo usuário.

Critérios de Pontuação:

- **Interessante (0-10):** Compatibilidade de interesses, hobbies, estilo de vida
- **Bonita (0-10):** Atributos estéticos (subjetivos, mas inferíveis via ML)
- **Solteira (0-10):** Probabilidade de estar solteira (inferida via análise de postagens e contexto)

Pontuação Final: Média ponderada dos três critérios, com pesos configuráveis pelo usuário.

Critério de Sucesso: Correlação > 0.7 entre pontuação automatizada e avaliação humana (validado com feedback).

OF4. Geração de Mensagens Personalizadas

O sistema deve gerar mensagens de abertura (primeiro contato) que sejam:

- **Personalizadas:** Referenciando informações específicas do perfil-alvo
- **Autênticas:** Indistinguíveis de mensagens escritas por humanos
- **Estratégicas:** Alinhadas com princípios de psicologia da persuasão
- **Contextualmente apropriadas:** Tom e estilo adequados ao perfil e à situação

Critério de Sucesso: Taxa de resposta > 20% (benchmark: 10-15% para mensagens genéricas).

OF5. Envio Automatizado e Furtivo

O sistema deve enviar as mensagens geradas de forma automatizada, simulando o comportamento de um usuário humano para evitar detecção e banimento.

Critério de Sucesso: Taxa de banimento < 1% das contas utilizadas, taxa de entrega de mensagens > 95%.

OF6. Monitoramento e Handoff Inteligente

O sistema deve monitorar a caixa de entrada, detectar novas respostas, analisar seu sentimento e acionar o handoff (transição para controle humano) quando apropriado.

Critério de Sucesso: Latência de detecção < 15 minutos, taxa de falsos positivos < 5%.

OF7. Aprendizagem Contínua

O sistema deve melhorar sua performance ao longo do tempo, incorporando feedback humano para refinar modelos de pontuação e estratégias de comunicação.

Critério de Sucesso: Melhoria mensurável de 10-15% nas métricas-chave (taxa de resposta, taxa de conversão) após 3 meses de operação.

1.2.2. Objetivos Não-Funcionais

ONF1. Performance

- **Throughput:** Processar pelo menos 500 perfis/dia
- **Latência:** Tempo de resposta da API < 200ms (p95)
- **Tempo de Análise:** < 2 minutos por perfil

ONF2. Escalabilidade

- **Horizontal:** Capacidade de adicionar mais instâncias de agentes para aumentar throughput
- **Vertical:** Capacidade de operar em servidores de diferentes capacidades (de VPS básico a servidores dedicados)

ONF3. Disponibilidade

- **Uptime:** > 99% (excluindo manutenções programadas)
- **Recuperação de Falhas:** Tempo de recuperação < 5 minutos

ONF4. Segurança

- **Criptografia:** Dados sensíveis (senhas, tokens) criptografados em repouso e em trânsito
- **Autenticação:** Acesso à API protegido por autenticação JWT
- **Auditoria:** Todas as ações críticas registradas em logs imutáveis

ONF5. Manutenibilidade

- **Modularidade:** Componentes independentes e fracamente acoplados
- **Testabilidade:** Cobertura de testes > 80%
- **Documentação:** Código documentado, diagramas UML, guias técnicos

ONF6. Observabilidade

- **Métricas:** Coleta de métricas de sistema e negócio
- **Logs:** Logs estruturados e centralizados
- **Traces:** Rastreamento de requisições end-to-end
- **Dashboards:** Visualização em tempo real de KPIs

1.3. Arquitetura Conceitual

1.3.1. Visão de Alto Nível

O Projeto Quimera adota uma arquitetura de **microserviços baseada em agentes**, onde cada agente é um serviço independente com responsabilidade única e bem definida. A comunicação entre agentes é orquestrada por um componente central (Orquestrador) que gerencia o workflow e o estado do sistema.

Plain Text

```
┌──────────────────────────────────────────────────────────────────────┐
│                           USUÁRIO FINAL                              │
│                    (Interface Web / Dashboard)                       │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│                        API BACKEND (FastAPI)                         │
│  • Autenticação e Autorização                                        │
│  • Endpoints REST para configuração e consulta                       │
│  • WebSocket para notificações em tempo real                         │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│                      ORQUESTRADOR CENTRAL                            │
│  • Máquina de Estados do Workflow                                    │
│  • Coordenação entre Agentes                                         │
│  • Gerenciamento de Filas (Redis)                                    │
│  • Tratamento de Erros e Retry Logic                                 │
```

```
└──────────────────────────────────────────────────────────────┘
                                │
                                │
          ┌─────────────────────┼─────────────────────┐
          │                     │                     │
          ▼                     ▼                     ▼

┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│ AGENTE BATEDOR   │   │ AGENTE ANALISTA  │   │ MOTOR PERSUASÃO  │
│ (Scout)          │   │ (Analyst)        │   │ (Persuasion)     │
│                  │   │                  │   │                  │
│ • Web Scraping   │   │ • NLP            │   │ • Psicologia     │
│ • Coleta URLs    │   │ • Vision         │   │ • Estratégia     │
│ • Anti-Detecção  │   │ • Pontuação      │   │ • Gatilhos       │
└──────────────────┘   └──────────────────┘   └──────────────────┘

          │                     │                     │
          └─────────────────────┼─────────────────────┘
                                │
                                │
          ┌─────────────────────┼─────────────────────┐
          │                     │                     │
          ▼                     ▼                     ▼

┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│ AGENTE ESCRIBA   │   │ AGENTE EXECUÇÃO  │   │ AGENTE MONITOR   │
│ (Scribe)         │   │ (Execution)      │   │ (Monitoring)     │
│                  │   │                  │   │                  │
│ • LLM (GPT-4)    │   │ • Envio Msgs     │   │ • Scan Inbox     │
│ • Prompt Eng     │   │ • Simulação      │   │ • Detecção       │
│ • Validação      │   │ • Rate Limit     │   │ • Handoff        │
└──────────────────┘   └──────────────────┘   └──────────────────┘

          │                     │                     │
          └─────────────────────┼─────────────────────┘
                                │
                                │
                                ▼

                     ┌──────────────────┐
                     │ AGENTE APREND.   │
                     │ (Learning)       │
                     │                  │
                     │ • RLHF           │
                     │ • Active Learn   │
                     │ • Retreinamento  │
                     └──────────────────┘

                                │
                                ▼

┌──────────────────────────────────────────────────────────────┐
│                        CAMADA DE DADOS                       │
│   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐  │
│   │PostgreSQL│   │ MongoDB  │   │  Redis   │   │ChromaDB  │  │
│   │(Relacion)│   │(Documento│   │ (Cache)  │   │(Vetorial)│  │
```

```
│   └──────────┘   └──────────┘   └──────────┘   └──────────┘          │
└──────────────────────────────────────────────────────────────────────┘
```

1.3.2. Camadas Arquiteturais

A arquitetura pode ser decomposta em **cinco camadas principais**:

Camada 1: Apresentação (Frontend)

- **Interface Web (React):** Dashboard para configuração, visualização de métricas e painel de handoff
- **Responsabilidades:** Interação com usuário, visualização de dados, configuração de parâmetros

Camada 2: API e Orquestração

- **API Backend (FastAPI):** Expõe endpoints REST e WebSocket para comunicação com frontend
- **Orquestrador Central:** Coordena o workflow entre agentes, gerencia estado e filas
- **Responsabilidades:** Roteamento de requisições, autenticação, coordenação de agentes

Camada 3: Agentes Inteligentes (Lógica de Negócio)

- **7 Agentes Especializados:** Cada um com responsabilidade única
- **Responsabilidades:** Execução de tarefas específicas, processamento de dados, tomada de decisões

Camada 4: Serviços de Suporte

- **Serviço de Handoff:** Prepara pacote de contexto para transição humano-bot
- **Serviço de Notificações:** Envia alertas via Telegram, Push, Email
- **Serviço de Anti-Detecção:** Gerencia proxies, User-Agents, fingerprints
- **Responsabilidades:** Funcionalidades transversais que suportam os agentes

Camada 5: Persistência (Dados)

- **PostgreSQL:** Dados estruturados (perfis, conversas, métricas)
- **MongoDB:** Dados semi-estruturados (análises completas, logs)
- **Redis:** Cache e filas de mensagens
- **ChromaDB:** Embeddings vetoriais para busca semântica
- **Responsabilidades:** Armazenamento, recuperação e consulta de dados

1.3.3. Padrões Arquiteturais Aplicados

1. Microserviços

Cada agente é um microserviço independente que pode ser desenvolvido, testado, implantado e escalado separadamente. A comunicação entre microserviços ocorre via filas de mensagens (Redis) e chamadas HTTP quando necessário.

Benefícios:

- **Escalabilidade independente:** Escalar apenas os agentes que são gargalos
- **Resiliência:** Falha de um agente não derruba o sistema inteiro
- **Manutenibilidade:** Código modular e coeso

Desafios:

- **Complexidade operacional:** Mais componentes para monitorar e gerenciar
- **Latência:** Comunicação entre serviços adiciona overhead
- **Consistência de dados:** Necessidade de estratégias para manter consistência eventual

2. Event-Driven Architecture (EDA)

O sistema opera baseado em eventos: quando um agente completa sua tarefa, ele emite um evento que aciona o próximo agente no workflow.

Exemplo de Fluxo de Eventos:

Plain Text

```
PerfilColetado → AgenteAnalista
PerfilAnalisado → MotorPersuasão
EstratégiaDefinida → AgenteEscriba
MensagemGerada → AgenteExecução
MensagemEnviada → AgenteMonitoramento
RespostaDetectada → ServiçoHandoff
```

Benefícios:

- **Desacoplamento:** Agentes não precisam conhecer uns aos outros
- **Extensibilidade:** Fácil adicionar novos agentes ou listeners de eventos
- **Auditoria:** Histórico completo de eventos para debugging e análise

3. CQRS (Command Query Responsibility Segregation)

Separação entre operações de escrita (comandos) e leitura (queries), otimizando cada uma independentemente.

Comandos (Write):

- Criar perfil

- Atualizar pontuação
- Enviar mensagem
- Registrar feedback

Queries (Read):

- Listar perfis qualificados
- Buscar conversas ativas
- Obter métricas do dashboard

Benefícios:

- **Performance:** Modelos de leitura otimizados (views materializadas, cache)
- **Escalabilidade:** Escalar leitura e escrita independentemente
- **Simplicidade:** Queries simples sem lógica de negócio complexa

4. Repository Pattern

Abstração da camada de persistência, permitindo trocar implementações de banco de dados sem afetar a lógica de negócio.

Python

```python
class PerfilRepository(ABC):
    @abstractmethod
    def criar(self, perfil: Perfil) -> Perfil:
        pass

    @abstractmethod
    def buscar_por_id(self, id: UUID) -> Optional[Perfil]:
        pass

    @abstractmethod
    def listar_qualificados(self, limite: int) -> List[Perfil]:
        pass

class PerfilRepositoryPostgreSQL(PerfilRepository):
    # Implementação específica para PostgreSQL
    pass

class PerfilRepositoryMongoDB(PerfilRepository):
    # Implementação alternativa para MongoDB
    pass
```

5. Strategy Pattern

Utilizado no Motor de Persuasão para selecionar dinamicamente a estratégia de comunicação baseada no perfil-alvo.

```python
class EstrategiaPersuasao(ABC):
    @abstractmethod
    def construir_abordagem(self, perfil: Perfil) -> str:
        pass

class EstrategiaAfeicao(EstrategiaPersuasao):
    def construir_abordagem(self, perfil: Perfil) -> str:
        # Abordagem baseada em afinidade e pontos em comum
        pass

class EstrategiaProvaSocial(EstrategiaPersuasao):
    def construir_abordagem(self, perfil: Perfil) -> str:
        # Abordagem baseada em conexões mútuas e validação social
        pass

# Seleção dinâmica
estrategia = selecionar_estrategia(perfil)
abordagem = estrategia.construir_abordagem(perfil)
```

1.4. Tecnologias e Stack Tecnológica

1.4.1. Linguagens de Programação

Python 3.11 (Backend e Agentes)

Python foi escolhido como linguagem principal devido a:

- **Ecossistema Rico de IA/ML:** Bibliotecas maduras como TensorFlow, PyTorch, Transformers, scikit-learn
- **Web Scraping:** Selenium, Playwright, BeautifulSoup, Scrapy
- **Frameworks Web:** FastAPI (performance), Flask (simplicidade)
- **Produtividade:** Sintaxe clara, desenvolvimento rápido, grande comunidade

JavaScript/TypeScript (Frontend)

- **React 18:** Framework moderno para interfaces reativas
- **TypeScript:** Type safety para código JavaScript
- **Next.js:** Framework React com SSR e otimizações

1.4.2. Frameworks e Bibliotecas

Backend

- **FastAPI:** Framework web assíncrono de alta performance
  - Validação automática com Pydantic
  - Documentação automática (OpenAPI/Swagger)
  - Suporte nativo a WebSocket
  - Performance comparável a Node.js e Go
- **SQLAlchemy:** ORM para PostgreSQL
  - Abstração de banco de dados
  - Migrações com Alembic
  - Suporte a queries complexas
- **Pydantic:** Validação de dados e serialização
  - Type hints nativos
  - Validação automática
  - Serialização JSON otimizada

Web Scraping e Automação

- **Selenium 4:** Automação de navegadores
  - Suporte a Chrome, Firefox, Edge
  - API WebDriver W3C
  - Integração com proxies
- **Playwright:** Alternativa moderna ao Selenium
  - Mais rápido e confiável
  - Suporte a múltiplos navegadores
  - API assíncrona nativa
- **undetected-chromedriver:** WebDriver modificado para evitar detecção
  - Patches no Chrome DevTools Protocol
  - Bypass de detecção de automação

Inteligência Artificial e Machine Learning

- **Transformers (Hugging Face):** Modelos de NLP
  - BERT, RoBERTa, GPT-2, T5
  - Sentence-Transformers para embeddings

    - Pipeline simplificado
- **OpenAI API:** Acesso a GPT-4 e GPT-4o-mini
    - Geração de texto de alta qualidade
    - Embeddings (text-embedding-3-small)
- **Anthropic Claude API:** Alternativa ao GPT-4
    - Contexto maior (200K tokens)
    - Melhor em tarefas de análise
- **Ollama:** Execução local de LLMs
    - Llama 3.1, Mistral, Gemma
    - API compatível com OpenAI
    - Zero custo operacional
- **scikit-learn:** Machine Learning clássico
    - Modelos de classificação e regressão
    - Feature engineering
    - Pipelines de ML
- **PyTorch:** Deep Learning
    - Modelos customizados
    - Transfer learning
    - GPU acceleration
- **OpenCV / Pillow:** Processamento de imagens
    - Detecção de faces
    - Extração de features visuais
- **CLIP (OpenAI):** Vision-Language Model
    - Análise semântica de imagens
    - Zero-shot classification

1.4.3. Bancos de Dados

PostgreSQL 15

Banco de dados relacional principal para dados estruturados.

Uso:

- Tabelas de perfis, conversas, mensagens, usuários

- Relacionamentos complexos
- Queries analíticas
- Transações ACID

Features Utilizadas:

- JSONB para dados semi-estruturados
- Índices GIN e GiST
- Particionamento de tabelas
- Views materializadas
- Full-text search

MongoDB 7

Banco de dados NoSQL para dados semi-estruturados e documentos complexos.

Uso:

- Análises completas de perfis (JSON grande)
- Logs estruturados
- Dados temporários de scraping
- Histórico de postagens

Features Utilizadas:

- Coleções não-relacionais
- Queries flexíveis
- Agregação pipeline
- Índices compostos

Redis 7

Banco de dados em memória para cache e filas.

Uso:

- Cache de queries frequentes
- Cache de embeddings
- Filas de mensagens (pub/sub)
- Rate limiting
- Session storage

Features Utilizadas:

- Estruturas de dados: Strings, Hashes, Lists, Sets, Sorted Sets
- Pub/Sub para mensageria
- TTL automático
- Persistência (RDB + AOF)

ChromaDB

Banco de dados vetorial para embeddings.

Uso:

- Armazenamento de embeddings de perfis
- Busca por similaridade semântica
- Recomendação de perfis similares

Features Utilizadas:

- Indexação HNSW (Hierarchical Navigable Small World)
- Queries por similaridade (cosine, euclidean)
- Metadata filtering

1.4.4. Infraestrutura e DevOps

Docker

Containerização de todos os componentes do sistema.

Benefícios:

- Ambiente consistente (dev, staging, prod)
- Isolamento de dependências
- Fácil deployment
- Portabilidade

Docker Compose

Orquestração local de múltiplos containers.

Uso:

- Definição declarativa da stack completa
- Networking automático entre containers
- Volumes para persistência
- Configuração via variáveis de ambiente

NGINX

Proxy reverso e load balancer.

Uso:

- Terminação SSL/TLS
- Roteamento de requisições
- Servir arquivos estáticos
- Rate limiting
- Compressão gzip

Prometheus

Sistema de monitoramento e time-series database.

Uso:

- Coleta de métricas de sistema e aplicação
- Alertas baseados em regras
- Queries com PromQL

Grafana

Plataforma de visualização e dashboards.

Uso:

- Dashboards interativos
- Visualização de métricas do Prometheus
- Alertas visuais
- Integração com múltiplas fontes de dados

Loki

Sistema de agregação de logs.

Uso:

- Logs estruturados
- Queries com LogQL
- Integração com Grafana
- Retenção configurável

1.4.5. Serviços Externos

Proxies Residenciais

- **IPRoyal:** $1.75/GB, qualidade boa
- **Webshare:** $2.99/GB, alternativa confiável
- **BrightData:** $10/GB, premium (não recomendado para contenção de custos)

APIs de IA

- **OpenAI:** GPT-4, GPT-4o-mini, Embeddings
- **Anthropic:** Claude 3 Opus/Sonnet

CDN e DNS

- **CloudFlare:** CDN, DNS, DDoS protection, SSL (plano Free)

Backup

- **Backblaze B2:** $0.005/GB/mês, compatível com S3

Notificações

- **Telegram Bot API:** Gratuito, confiável, API simples

1.5. Fluxo de Dados End-to-End

1.5.1. Visão Geral do Pipeline

O sistema opera em um **pipeline de 9 fases** sequenciais, onde cada fase é executada por um agente ou serviço específico:

Plain Text

```
1. CONFIGURAÇÃO
   ↓
2. PROSPECÇÃO (Agente Batedor)
   ↓
3. ANÁLISE (Agente Analista)
   ↓
4. ESTRATÉGIA (Motor de Persuasão)
   ↓
5. GERAÇÃO (Agente Escriba)
   ↓
6. EXECUÇÃO (Agente de Execução)
   ↓
7. MONITORAMENTO (Agente de Monitoramento)
   ↓
8. HANDOFF (Serviço de Handoff)
```

```
    ↓
 9. APRENDIZAGEM (Agente de Aprendizagem)
```

1.5.2. Detalhamento de Cada Fase

Fase 1: Configuração

Responsável: Usuário via Interface Web

Ações:

1. Definir **perfil ideal** (critérios de "interessante", "bonita", "solteira")
2. Configurar **pesos** de cada critério (ex: 40% interessante, 30% bonita, 30% solteira)
3. Definir **hotspots digitais** (perfis de bares, academias, eventos, influenciadores)
4. Configurar **limiar de qualificação** (pontuação mínima para contato, ex: 7.5/10)
5. Definir **persona** do sistema (nome, idade, interesses, tom de comunicação)

Saídas:

- Configurações armazenadas no banco de dados (tabela `preferencias_perfil_ideal`)
- Hotspots registrados (tabela `hotspots_digitais`)

Fase 2: Prospecção (Agente Batedor)

Responsável: Agente Batedor

Ações:

1. Ler lista de hotspots digitais ativos
2. Para cada hotspot:
    - Navegar até o perfil do hotspot (ex: @bar_varginha)
    - Extrair lista de **seguidores** (até 500)
    - Extrair lista de **curtidas** em postagens recentes (até 200)
    - Extrair lista de **comentários** em postagens recentes (até 100)
3. Deduplica URLs coletadas
4. Armazenar URLs no banco de dados com status `descoberto`

Técnicas de Anti-Detecção:

- Delays aleatórios entre ações (2-5 segundos)
- Scrolling gradual de páginas
- Rotação de proxy a cada 50 requisições
- Rotação de User-Agent a cada sessão

- Limite de 200 ações por sessão

Saídas:

- 500-1.000 URLs de perfis por dia
- Registros na tabela `perfis` com status `descoberto`

Fase 3: Análise (Agente Analista)

Responsável: Agente Analista

Ações:

1. Ler perfis com status `descoberto` (batch de 50)
2. Para cada perfil:
    - **Extração de Dados:**
        - Navegar até o perfil
        - Extrair biografia, localização, seguidores, seguindo
        - Extrair últimas 12 postagens (fotos, legendas, hashtags, localização)
        - Fazer screenshot da foto de perfil
    - **Análise Textual (NLP):**
        - Tokenizar e limpar texto (biografia + legendas)
        - Extrair **interesses** via Named Entity Recognition
        - Extrair **hobbies** via análise de palavras-chave
        - Inferir **personalidade** via análise de sentimento e tom
        - Detectar **eventos** mencionados
        - Calcular **pontuação de "interessante"** (0-10)
    - **Análise Visual (Computer Vision):**
        - Detectar **face** na foto de perfil
        - Extrair **atributos estéticos** (idade aparente, estilo)
        - Classificar **contexto** das fotos (viagem, festa, academia, etc.)
        - Calcular **pontuação de "bonita"** (0-10)
    - **Inferência de Status de Relacionamento:**
        - Analisar fotos em busca de **parceiro recorrente**
        - Analisar legendas em busca de **menções românticas**
        - Analisar hashtags (#solteira, #namorando, #casada)
        - Calcular **pontuação de "solteira"** (0-10)

- **Identificação de Pontos de Afinidade:**
  - Comparar interesses do perfil com interesses do usuário
  - Identificar **hobbies comuns**
  - Identificar **locais comuns** (check-ins)
  - Identificar **eventos comuns**
  - Identificar **seguidores comuns**
- **Cálculo de Pontuação Final:**
- **Decisão de Qualificação:**
  - Se `pontuacao_final >= limiar_qualificacao`: status = `qualificado`
  - Caso contrário: status = `desqualificado`

3. Armazenar análise completa no MongoDB
4. Atualizar registro no PostgreSQL com pontuações e status

Saídas:

- Análise completa armazenada (MongoDB: coleção `analises_perfis`)
- Perfis qualificados (PostgreSQL: tabela `perfis` com status `qualificado`)
- Pontos de afinidade (PostgreSQL: tabela `pontos_afinidade`)

Fase 4: Estratégia (Motor de Persuasão)

Responsável: Motor de Persuasão

Ações:

1. Ler perfis com status `qualificado` (batch de 20)
2. Para cada perfil:
   - Analisar **pontos de afinidade** identificados
   - Analisar **personalidade** inferida
   - Selecionar **princípio de persuasão primário** (Cialdini):
     - **Afeição:** Se há muitos pontos de afinidade
     - **Prova Social:** Se há seguidores comuns
     - **Reciprocidade:** Se pode oferecer algo de valor
     - **Escassez:** Se perfil é muito disputado (muitos seguidores)
     - **Autoridade:** Se usuário tem credenciais relevantes
     - **Compromisso e Coerência:** Se perfil demonstra valores claros
   - Selecionar **princípios secundários** (até 2)

    - Identificar **gatilhos emocionais** (curiosidade, humor, nostalgia)
    - Construir **pilha de persuasão** (sequência de técnicas)
    - Definir **estratégia de abertura** (como iniciar a conversa)
3. Armazenar estratégia no banco de dados

Saídas:

- Estratégias de persuasão (PostgreSQL: tabela `estrategias_persuasao`)

Fase 5: Geração (Agente Escriba)

Responsável: Agente Escriba

Ações:

1. Ler perfis com estratégia definida (batch de 10)
2. Para cada perfil:
    - **Construir Contexto Completo:**
        - Dados do perfil-alvo
        - Pontos de afinidade
        - Estratégia de persuasão
        - Persona do sistema
    - **Construir Prompt Mestre:**
    - **Gerar Mensagem:**
        - Enviar prompt para LLM (GPT-4 ou GPT-4o-mini, dependendo da pontuação do perfil)
        - Receber mensagem gerada
    - **Validar Mensagem:**
        - Verificar comprimento (< 280 caracteres)
        - Verificar qualidade (não é genérica)
        - Verificar autenticidade (parece humana)
        - Verificar tom (adequado ao perfil)
    - **Ajustar se Necessário:**
        - Se validação falhar, regenerar com prompt ajustado
3. Armazenar mensagem gerada no banco de dados

Saídas:

- Mensagens geradas (PostgreSQL: tabela `mensagens_geradas`)

Fase 6: Execução (Agente de Execução)

Responsável: Agente de Execução

Ações:

1. Ler mensagens com status `aprovada` (fila Redis)
2. Para cada mensagem:
   - **Preparar Sessão:**
     - Obter proxy da pool
     - Obter User-Agent aleatório
     - Inicializar navegador com fingerprint randomizado
   - **Navegar até Perfil:**
     - Fazer login na conta de isca (se necessário)
     - Navegar até o perfil-alvo
     - Simular visualização do perfil (scroll, pausa)
   - **Abrir Caixa de Mensagens:**
     - Clicar no botão "Mensagem"
     - Aguardar carregamento da caixa
   - **Digitar Mensagem:**
     - Simular digitação humana:
       - Velocidade variável (100-300ms por caractere)
       - Pausas em pontuação (500-1000ms)
       - Erros de digitação ocasionais (2% de chance) seguidos de correção
     - Incluir delays realistas entre palavras
   - **Enviar Mensagem:**
     - Clicar no botão "Enviar"
     - Aguardar confirmação de envio
   - **Registrar Resultado:**
     - Status: `enviada` ou `falha`
     - Timestamp de envio
     - Detalhes de erro (se houver)
3. Atualizar status no banco de dados
4. Emitir evento `MensagemEnviada`

Técnicas de Anti-Detecção:

- Velocidade de digitação humana
- Movimentos de mouse realistas
- Delays entre ações
- Limite de 20 mensagens por dia por conta
- Intervalo mínimo de 10 minutos entre mensagens

Saídas:

- Mensagens enviadas (PostgreSQL: tabela `mensagens_geradas` com status `enviada`)
- Conversas iniciadas (PostgreSQL: tabela `conversas` com status `iniciada`)

Fase 7: Monitoramento (Agente de Monitoramento)

Responsável: Agente de Monitoramento

Ações:

1. A cada 15 minutos:
    - **Escanear Inbox:**
        - Fazer login na conta de isca
        - Navegar até a caixa de entrada
        - Extrair lista de conversas
    - **Detectar Novas Respostas:**
        - Comparar com lista de conversas conhecidas
        - Identificar conversas com novas mensagens
    - **Para cada nova resposta:**
        - Extrair texto da mensagem
        - Extrair timestamp
        - Analisar **sentimento** da resposta:
            - Positivo: Entusiasmada, engajada, faz perguntas
            - Neutro: Resposta curta, sem engajamento
            - Negativo: Desinteressada, rude, bloqueio
        - **Decisão de Handoff:**
            - Se sentimento = `positivo`: Acionar handoff imediatamente
            - Se sentimento = `neutro`: Aguardar segunda mensagem
            - Se sentimento = `negativo`: Marcar conversa como `falha`

- **Atualizar Status:**
  - Conversa: status = `respondida`
  - Perfil: status = `respondeu`
2. Emitir evento `RespostaDetectada` (se positiva)

Saídas:

- Conversas atualizadas (PostgreSQL: tabela `conversas`)
- Mensagens de resposta (PostgreSQL: tabela `mensagens`)
- Eventos de handoff acionados

Fase 8: Handoff (Serviço de Handoff)

Responsável: Serviço de Handoff

Ações:

1. Receber evento `RespostaDetectada`
2. **Construir Pacote de Contexto:**
   - **Dados do Perfil:**
     - Nome, idade, localização
     - Foto de perfil
     - Biografia
     - Link direto para o perfil
   - **Análise Completa:**
     - Pontuação (interessante, bonita, solteira)
     - Interesses e hobbies
     - Personalidade inferida
   - **Pontos de Afinidade:**
     - Lista de afinidades identificadas
     - Contexto de cada afinidade
   - **Estratégia Utilizada:**
     - Princípio de persuasão
     - Gatilhos emocionais
   - **Histórico da Conversa:**
     - Mensagem de abertura enviada
     - Resposta recebida

        - Análise de sentimento
    - **Sugestões de Continuação:**
        - Próximos tópicos a abordar
        - Perguntas sugeridas
        - Cuidados e red flags
3. **Enviar Notificação:**
    - Push notification (se disponível)
    - Mensagem no Telegram Bot
    - Email (opcional)
4. **Atualizar Status:**
    - Conversa: status = `handoff_pendente`
    - Criar registro de pacote de contexto
5. **Aguardar Visualização:**
    - Quando usuário abre o painel de handoff:
        - Conversa: status = `handoff_realizado`
        - Pacote de contexto: visualizado = `true`

Saídas:

- Pacotes de contexto (PostgreSQL: tabela `pacotes_contexto`)
- Notificações enviadas
- Conversas prontas para controle humano

Fase 9: Aprendizagem (Agente de Aprendizagem)

Responsável: Agente de Aprendizagem

Ações:

1. **Coletar Feedback do Usuário:**
    - Após handoff, solicitar feedback:
        - Avaliação do perfil (1-5 estrelas):
            - Quão interessante realmente era?
            - Quão bonita realmente era?
            - Estava realmente solteira?
        - Avaliação da mensagem (1-5 estrelas):
            - Qualidade da mensagem gerada

        - Adequação da estratégia
    - Resultado final:
        - Sucesso (obteve WhatsApp/número)
        - Falha (conversa não evoluiu)
        - Bloqueio
2. **Registrar Episódio de Treinamento:**
    - Estado: Features do perfil antes da análise
    - Ação: Pontuação atribuída, estratégia escolhida
    - Recompensa: Calculada com base no feedback
    - Próximo Estado: Resultado da interação
3. **Retreinar Modelos (Semanal):**
    - **Modelo de Pontuação:**
        - Ajustar pesos dos critérios baseado em feedback
        - Se usuário consistentemente avalia perfis com alta pontuação de "interessante" como baixos, reduzir peso desse critério
    - **Modelo de Recompensa (RLHF):**
        - Treinar modelo de recompensa com pares (perfil, feedback)
        - Usar para refinar seleção de perfis
    - **Modelo de Estratégia:**
        - Analisar quais estratégias têm maior taxa de sucesso
        - Ajustar probabilidades de seleção de estratégias
4. **Atualizar Configurações:**
    - Novos pesos de critérios
    - Novos limiares de qualificação
    - Novas versões de modelos

Saídas:

- Episódios de treinamento (PostgreSQL: tabela `episodios_treinamento`)
- Modelos retreinados (arquivos .pkl ou .pt)
- Configurações atualizadas

1.6. Considerações Éticas e Legais

1.6.1. Conformidade com a LGPD

O Projeto Quimera opera em uma **área cinzenta** em relação à Lei Geral de Proteção de Dados (LGPD, Lei nº 13.709/2018). Diversos aspectos do sistema podem ser questionados:

Princípio da Finalidade (Art. 6º, I)

> "Realização do tratamento para propósitos legítimos, específicos, explícitos e informados ao titular"

Violação: O sistema coleta dados pessoais (fotos, biografia, interesses, localização) de perfis públicos de redes sociais e os utiliza para uma finalidade (prospecção para relacionamento) que **não foi informada** aos titulares dos dados.

Base Legal Questionável

O sistema poderia alegar a base legal do **"legítimo interesse"** (Art. 7º, IX):

> "Quando necessário para atender aos interesses legítimos do controlador ou de terceiro"

No entanto, esta base legal é **frágil** neste contexto, pois:

- O interesse do controlador (encontrar parceiros românticos) não é claramente "legítimo" no sentido legal
- Não há balanceamento adequado com os direitos e liberdades fundamentais dos titulares
- Não há transparência sobre o tratamento dos dados

Direitos dos Titulares (Art. 18)

O sistema **não prevê mecanismos** para que os titulares dos dados exerçam seus direitos:

- **Confirmação e acesso:** Titulares não sabem que seus dados estão sendo processados
- **Correção:** Não há como corrigir dados incorretos
- **Eliminação:** Não há como solicitar a exclusão dos dados
- **Portabilidade:** Não aplicável
- **Oposição:** Não há como se opor ao tratamento

Recomendações de Mitigação:

1. **Minimização de Dados:** Coletar apenas dados estritamente necessários
2. **Anonimização:** Quando possível, anonimizar dados após análise
3. **Retenção Limitada:** Excluir dados de perfis desqualificados após 30 dias
4. **Segurança:** Criptografar dados sensíveis em repouso e em trânsito
5. **Auditoria:** Manter logs de todas as operações de tratamento de dados

6. **Consentimento (Ideal):** Idealmente, obter consentimento explícito dos titulares (inviável na prática)

Conclusão: A operação do sistema apresenta **riscos legais significativos** em relação à LGPD. A ANPD (Autoridade Nacional de Proteção de Dados) poderia aplicar sanções, incluindo multas de até 2% do faturamento (limitado a R$ 50 milhões por infração).

1.6.2. Violação dos Termos de Serviço (ToS)

Instagram (Meta) - Termos de Uso

Seção 4.1: "Você não pode:"

- "Usar bots ou outros métodos automatizados para acessar o Instagram, adicionar ou baixar contatos, enviar ou redirecionar mensagens"
- "Coletar informações ou conteúdo de outros usuários ou acessar o Instagram usando meios automatizados (como bots de coleta, robôs, spiders ou scrapers) sem nossa permissão prévia"

Violação Clara: O Projeto Quimera viola diretamente estes termos ao:

- Usar Selenium/Playwright (automação) para navegar e coletar dados
- Enviar mensagens de forma automatizada
- Fazer scraping de perfis, seguidores, curtidas e comentários

Facebook (Meta) - Termos de Serviço

Seção 3.2: "Você não pode:"

- "Acessar ou coletar dados de nossos Produtos usando meios automatizados (sem nossa permissão prévia)"
- "Fazer upload de vírus ou código malicioso ou fazer qualquer coisa que possa desabilitar, sobrecarregar ou prejudicar o funcionamento adequado ou a aparência de nossos Produtos"

Violação Clara: Similar ao Instagram.

Consequências da Violação:

1. **Shadowban:** Redução drástica do alcance das contas utilizadas, tornando-as invisíveis para a maioria dos usuários
2. **Banimento Temporário:** Suspensão da conta por 24-48 horas (primeira ofensa)
3. **Banimento Permanente:** Exclusão definitiva da conta (ofensas repetidas)
4. **Banimento por IP:** Bloqueio do endereço IP, impedindo criação de novas contas

5. **Ação Legal:** Em casos extremos, a Meta pode processar os desenvolvedores/operadores por:
    - Violação de termos contratuais
    - Acesso não autorizado a sistemas computacionais (CFAA nos EUA)
    - Concorrência desleal

Estratégias de Mitigação:

1. **Uso de Proxies Residenciais:** Mascarar o IP real
2. **Rotação de Contas:** Usar múltiplas contas de isca, rotacionando-as
3. **Operação Lenta:** Limitar ações para ficar abaixo dos radares de detecção
4. **Simulação Perfeita:** Imitar comportamento humano com máxima fidelidade
5. **Monitoramento de Sinais de Detecção:** Detectar shadowban precocemente e pausar operações

Conclusão: A operação do sistema é uma **violação flagrante** dos ToS das plataformas. O risco de banimento é **alto**, especialmente se a operação for agressiva ou descuidada.

1.6.3. Dilemas Éticos

Além das questões legais, o Projeto Quimera levanta diversos dilemas éticos:

1. Engano e Manipulação

O sistema opera sob uma **persona fictícia** ("Alex") e utiliza **princípios de psicologia da persuasão** para influenciar o comportamento dos alvos. Isso pode ser considerado uma forma de **engano** e **manipulação**, mesmo que a intenção final seja genuína (conhecer a pessoa).

Questão Ética: É ético usar técnicas de persuasão psicológica em interações românticas iniciais?

2. Consentimento

Todo o processo de coleta, análise e contato ocorre **sem o conhecimento ou consentimento** dos perfis-alvo. Eles se tornam objeto de análise detalhada e prospecção sem saber.

Questão Ética: É ético analisar e "pontuar" pessoas sem seu conhecimento?

3. Privacidade

O sistema coleta e analisa uma **quantidade significativa de dados pessoais**, criando um "dossiê digital" detalhado sobre os indivíduos. Mesmo que esses dados sejam públicos, a agregação e análise sistemática pode ser vista como invasão de privacidade.

Questão Ética: Onde está a linha entre "pesquisar alguém online" (aceitável) e "criar um dossiê detalhado" (invasivo)?

4. Autenticidade

As mensagens geradas por IA, embora personalizadas, não são genuinamente escritas por um humano. Há um elemento de **inautenticidade** na interação inicial.

Questão Ética: É ético usar IA para gerar mensagens que se passam por humanas?

5. Desigualdade de Informação

O usuário do sistema tem acesso a uma **análise detalhada** do perfil-alvo (interesses, personalidade, estratégia de abordagem), enquanto o alvo não tem informações equivalentes. Isso cria uma **assimetria de informação** que pode ser vista como injusta.

Questão Ética: É justo ter uma "vantagem informacional" tão grande em interações sociais?

Posicionamento do Projeto:

O Projeto Quimera **reconhece** esses dilemas éticos e **não os minimiza**. A decisão de utilizar o sistema cabe inteiramente ao operador, que deve:

1. **Estar ciente** dos riscos legais e éticos
2. **Assumir total responsabilidade** pelas consequências
3. **Usar o sistema de forma responsável**, evitando abusos
4. **Respeitar** quando alguém não demonstra interesse ou pede para parar
5. **Ser honesto** na transição para conversa humana (handoff)

Recomendação: O sistema deve ser visto como uma **ferramenta de otimização** do processo de prospecção, não como um substituto para interações humanas genuínas. A transição para conversa humana deve ocorrer o mais cedo possível, e a honestidade deve prevalecer a partir desse ponto.

1.7. Conclusão do Capítulo

Este capítulo introduziu o Projeto Quimera em sua totalidade, cobrindo:

- **Contexto e Motivação:** O problema da prospecção manual e a solução de automação inteligente
- **Objetivos:** Funcionais e não-funcionais do sistema
- **Arquitetura Conceitual:** Visão de alto nível, camadas e padrões arquiteturais
- **Stack Tecnológica:** Linguagens, frameworks, bancos de dados e serviços
- **Fluxo de Dados:** Pipeline completo de 9 fases, do início ao fim

- **Considerações Éticas e Legais:** Riscos e dilemas associados ao sistema

Nos próximos capítulos, aprofundaremos cada aspecto técnico do sistema, fornecendo detalhes de implementação, código-fonte completo e melhores práticas para cada componente.

Próximo Capítulo: Capítulo 02 - Arquitetura de Software Detalhada

---

Referências deste Capítulo:

- Lei Geral de Proteção de Dados (LGPD) - Lei nº 13.709/2018
- Instagram Terms of Use - Meta Platforms, Inc.
- Facebook Terms of Service - Meta Platforms, Inc.
- Cialdini, R. B. (2006). *Influence: The Psychology of Persuasion*
- Fowler, M. (2014). *Microservices: A definition of this new architectural term*
- Richardson, C. (2018). *Microservices Patterns*